CODEGEN RAG CAPSTONE

Evaluating and Enhancing Code Language Models with Retrieval-Augmented Generation

Full Capstone Report — Project and Delivery

AIML PGCP Capstone

Group Number: 39 · Batch: 26

TalentSprint / Accenture

Team Members:

Mahin Nandipa · Abhinaya Thavishi · Ashu Bagul

Mentors: Pawan Baswani, Nayan Anand

Supervisor: Prof. Anil Neelkanti

August 2026

Table of Contents

Abstract

This report is a complete account of the CodeGen RAG Capstone project: the research system built across four checkpoints, and the engineering work carried out to synchronize, repair, test, deploy, and publish that system. The underlying project evaluates and enhances Salesforce/codegen-350M-multi, a small open-source code language model, across five software-engineering tasks — program synthesis, documentation generation, commit message generation, code translation, and natural-language-to-SQL generation — comparing it against a Rust-fine-tuned variant, an upper-bound LLM (Claude Sonnet 4 with GPT-5 fallback), and a Retrieval-Augmented Generation pipeline layered on that upper-bound LLM.

Beyond the original four-checkpoint research scope, this report documents a substantial delivery-hardening phase: repository synchronization across a divergent sandbox and live Drive copy, two functional bug fixes (checkpoint-aware model routing and genuine AST-aware CodeBLEU scoring), three infrastructure issues root-caused and resolved, a Checkpoint 4 requirement gap closed at the code level and subsequently verified live in production, a new live per-query scoring feature added to the deployed demo, test coverage grown from 108 to 164 passing tests, and publication to two GitHub repositories with a durable, always-on deployment path identified for the public demo. Every claim of completion in this report is backed by a specific, checkable artifact — a test count, a rendered PDF, a live screenshot, or a git commit hash — rather than an unverified assertion.

1. Problem Statement

1.1 Background and Motivation

Code language models have become a standard part of the software engineering toolchain, from inline autocomplete to full pull-request generation. The strongest of these systems are large, proprietary, and expensive to run at scale: every inference call typically leaves an organization's infrastructure and incurs a per-token cost. At the other end of the spectrum sit small, openly licensed code models — hundreds of millions of parameters rather than tens or hundreds of billions — that can be self-hosted, fine-tuned cheaply, and run entirely within an organization's own infrastructure. The practical question most engineering teams actually face is not "which model scores highest on a public leaderboard" but "is a small, self-hosted model good enough for our specific task, and if not, what is the cheapest way to close the gap."

Two techniques are commonly proposed to close that gap without simply switching to a larger, more expensive model: targeted fine-tuning on a narrow domain, and retrieval augmentation that grounds generation in relevant external content at inference time rather than requiring that content to be memorized during pretraining. Both techniques are well studied individually in the literature (see Section 2), but a project that measures them side by side, on the same tasks, with the same evaluation harness, and pushes the result through a real deployment is comparatively rare in a coursework setting — most classroom projects stop at a notebook with printed metrics. This project was scoped specifically to go further than that: to build something that could be deployed, demonstrated live, and handed to another engineer to run.

1.2 Problem Definition

This project addresses two concrete, measurable questions. First, how much of the capability gap between a 350-million-parameter open model and both a much larger proprietary model can be closed, for a single

underrepresented target language, through parameter-efficient and full fine-tuning alone, without changing the base model's parameter count or architecture. Rust was chosen as that target language specifically because it is absent from codegen-350M-multi's original pretraining corpus, making any improvement attributable to the fine-tuning step rather than to prior exposure. Second, how much additional generation quality can be recovered — for both the small base model and the upper-bound LLM — by augmenting generation with retrieval over a relevant code corpus at inference time, and whether combining dense semantic retrieval with structural, AST-based retrieval outperforms either technique used alone.

1.3 Objectives

- Evaluate codegen-350M-multi as a baseline across program synthesis, documentation generation, commit message generation, code translation, and text-to-SQL generation, using automatic metrics validated against human-judgment-correlated methods (CodeBLEU, CodeBERTScore, execution accuracy, pass@k).
- Extend the base model to Rust via parameter-efficient (LoRA) and full fine-tuning, and route deployment traffic to the best available fine-tuned checkpoint automatically, per language, without manual intervention.
- Build a Retrieval-Augmented Generation pipeline combining dense embedding search (FAISS) and AST-structural search, fused via reciprocal rank fusion, and measure its effect across all four model tiers (small baseline, fine-tuned, upper-bound LLM, and upper-bound LLM plus RAG).
- Deploy the complete system — base model, fine-tuned model, RAG pipeline, and upper-bound LLM comparison — behind a documented, containerized, and tested API and demo UI, then verify and publish that system for team and mentor review.
- Beyond the original checkpoint scope: audit the finished system honestly against the official project brief, close any gaps found at the code level, verify those closures with real evidence rather than assumption, and leave the project in a state a grader or teammate can run end to end without additional guidance.

1.4 Scope and Non-Goals

This project targets a single new language (Rust) for the fine-tuning objective, rather than a broad multilingual extension — the fine-tuning methodology is designed to generalize (Section 8.2), but only Rust was actually executed within the project timeline. Retrieval augmentation is evaluated over a general-purpose code corpus (CodeParrot plus CoDocBench) rather than a specific organization's proprietary codebase, since no such codebase was available for this coursework context; Section 8.4 discusses how the same pipeline transfers to that setting. Finally, the project deliberately does not attempt to train a new base model from scratch, or to reproduce every ablation an academic paper on this topic might include — the goal throughout was a working, evaluated, deployed system within a fixed four-checkpoint timeline, not an exhaustive research study.

2. Literature Review

This project builds directly on twelve prior works, summarized in full (with corrected citation details) in the companion Research Paper Summaries document. This section reviews each in turn, then explains how they combine into this project's specific design.

2.1 Base Model and Generation Framework — CodeGen

CodeGen (Nijkamp et al., 2022) introduces a family of open, autoregressive code language models trained with a multi-turn program synthesis framing: rather than generating a full program from a single prompt, the model is trained to incrementally refine a partial program across a conversational sequence of natural-language instructions. This project uses the smallest publicly released variant, codegen-350M-multi, trained on a multilingual mixture of natural language (ThePile) and source code (BigQuery, BigPython). Choosing the smallest variant was deliberate: it makes the base model's limitations visible and worth improving on, and keeps every experiment runnable on a single consumer-grade GPU, which matters directly for this project's cost-efficiency argument in Section 8.1.

2.2 Functional Correctness Evaluation — Codex and pass@k

Chen et al. (2021) introduces both the Codex model (the system underlying early GitHub Copilot) and the pass@k metric, which estimates the probability that at least one of k independently sampled completions passes a held-out functional test suite. Pass@k directly measures what actually matters for a code-generation system — does the code work — rather than how closely it resembles a reference solution textually. This project reports pass@k where the task and sampling budget allow (primarily program synthesis), while relying on the text-similarity metrics described in Section 2.7 for tasks such as documentation and commit-message generation where no single "correct" functional output exists to test against.

2.3 Semantic Code Understanding — CodeBERT

CodeBERT (Feng et al., 2020) is a bimodal pretrained transformer for programming and natural languages, trained jointly on code and paired natural-language documentation. It established the general pattern this project's dense retrieval design follows: encode both queries and candidate code chunks into a shared embedding space, then retrieve by vector similarity. CodeBERT's encoder-based approach also underlies CodeBERTScore-style semantic similarity metrics, discussed further in Section 2.7, which this project uses as a soft complement to CodeBLEU's structural scoring.

2.4 Documentation and Maintenance — CoDocBench

CoDocBench (Pai et al., 2024 — corrected from "Pal et al." as it appears in the project README) is a dataset of paired code-and-documentation snapshots mined from real commit history, specifically designed to evaluate whether a model can generate documentation that stays synchronized with code as it evolves, rather than documentation for a static, isolated snippet. This project uses CoDocBench both as a documentation-generation evaluation set and, separately, as part of the retrieval corpus for the RAG pipeline described in Section 3.5, since its paired code/documentation structure is directly useful as retrievable context.

2.5 Text-to-SQL Benchmarks — Spider and BIRD

Two complementary text-to-SQL benchmarks inform the SQL generation evaluation in this project. Spider (Yu et al., 2018) is a cross-domain benchmark spanning 200 databases across many domains, specifically constructed so that a model cannot succeed by memorizing schema-specific patterns — every evaluation database is unseen during training. BIRD (Li et al., 2023 — corrected from "2024" as it appears in the project README) tests a harder, more realistic setting: large, messy, real-world production databases with dirty values, ambiguous column names, and external domain knowledge often required to write a correct query. Reporting both benchmarks together, rather than either alone, gives a more complete picture of a

model's text-to-SQL capability — Spider measures generalization across schemas, BIRD measures robustness against realistic data quality.

2.6 Retrieval-Augmented Generation — Lewis et al. and ReACC

Lewis et al. (2020) establishes the general Retrieval-Augmented Generation pattern this project's RAG pipeline follows: combine a parametric generator (the language model) with a non-parametric retrieval memory (a searchable corpus), retrieving relevant content at inference time rather than requiring it to be memorized during pretraining. The closest prior work specifically applying this pattern to code is ReACC (Lu et al., 2021), a retrieval-augmented code completion framework combining sparse (lexical) and dense (semantic) retrieval. This project extends that combination by adding a third retrieval signal — AST-structural retrieval — fused with dense retrieval via reciprocal rank fusion, described fully in Section 3.5.

2.7 Automatic Evaluation Metrics — CodeBLEU and CodeBERTScore

Two complementary automatic metrics anchor this project's evaluation harness beyond simple exact-match and pass@k. CodeBLEU (Ren et al., 2020) extends the standard BLEU n-gram overlap metric with two code-specific components — weighted keyword matching and, where a language grammar is available, AST structural match and dataflow match — making it sensitive to syntactic and structural correctness in a way plain text BLEU is not. CodeBERTScore (Zhou et al., 2023) instead scores semantic similarity via contextual embeddings, making it more forgiving of superficial rewording (variable renaming, equivalent control-flow restructuring) that would otherwise depress an n-gram-based score even for a functionally equivalent output. This project reports both together specifically because they can diverge in informative ways — Section 5.4 discusses a concrete example where this divergence changes how a result should be interpreted.

2.8 Synthesis and Positioning of This Project

Positioned against this literature, this project's specific contribution is not a new technique but a controlled, side-by-side comparison across four model tiers on identical tasks with an identical evaluation harness, carried through to a real, tested, and deployed system rather than stopping at offline metrics. Where CodeGen supplies the base model, Codex/pass@k and CodeBLEU/CodeBERTScore supply the evaluation methodology, Spider/BIRD/CoDocBench supply the task-specific benchmarks, and Lewis et al./ReACC supply the retrieval-augmentation pattern, this project's engineering contribution is combining all of them into one coherent, testable, deployable pipeline — plus the hybrid dense-and-AST retrieval fusion, which is not present as a combination in any single one of the reviewed papers.

Literature-to-project mapping

Prior Work	What It Contributes	How This Project Uses It
CodeGen (Nijkamp et al., 2022)	Open, multi-turn code generation model family	Supplies the base model (codegen-350M-multi) evaluated and fine-tuned throughout
Codex / pass@k (Chen et al., 2021)	Functional-correctness sampling metric	Reported for program synthesis where sampling budget allows
CodeBERT (Feng et al., 2020)	Bimodal code/NL embedding pattern	Pattern reused for dense retrieval embeddings and semantic scoring

Prior Work	What It Contributes	How This Project Uses It
CoDocBench (Pai et al., 2024)	Paired code/documentation dataset	Documentation-generation evaluation set and RAG retrieval corpus
Spider (Yu et al., 2018)	Cross-domain text-to-SQL benchmark	SQL generation evaluation (Section 5.1)
BIRD (Li et al., 2023)	Realistic, messy-database text-to-SQL benchmark	SQL generation evaluation, harder-setting comparison to Spider
Lewis et al. (2020) / ReACC (Lu et al., 2021)	Retrieval-augmented generation pattern for text and code	Backbone design pattern for the project's RAG pipeline
CodeBLEU (Ren et al., 2020)	Structure-aware code similarity metric	Primary similarity metric across all non-SQL, non-exact-match evaluations
CodeBERTScore (Zhou et al., 2023)	Embedding-based semantic similarity metric	Reported alongside CodeBLEU to catch semantically valid but structurally different output

2.9 Spider vs. Spider 2.0 — A Resolved Discrepancy

The official project brief's reference list links to spider2-sql.github.io — Spider 2.0, a substantially newer and harder benchmark built against live enterprise data warehouses (BigQuery, Snowflake, DuckDB) rather than a fixed set of downloadable SQLite databases. This project's actual implementation was checked directly against its source code for this report: src/codegen_rag/data/downloaders.py downloads the xlangai/spider dataset from Hugging Face — the classic 2018 Spider benchmark (Yu et al.), with 200 cross-domain databases, matching the citation already used in the project README and throughout this report's results.

This is very likely a reference-list error in the brief rather than a defect in the project, since classic Spider is what "Spider" conventionally refers to in this kind of coursework and Spider 2.0's live-warehouse setup would require an entirely different, much larger-scope data and evaluation pipeline (live cloud warehouse credentials, a different query dialect surface, and no fixed downloadable database archive). The recommendation carried into this report's Limitations section (11.2) is a one-line confirmation with the mentors that classic Spider was the intended benchmark, rather than a rebuild against Spider 2.0.

3. Methodology

3.1 System Architecture Overview

The system is organized into six layers that share a single typed Settings object end to end, so that all four project checkpoints operate on one coherent pipeline rather than four disconnected demos. From bottom to top: a data layer (downloaders, cleaners, preprocessors), a model layer (the base model wrapper and

generation configuration), a task layer (five thin task modules sharing a common base class), a training layer (LoRA and full fine-tuning, with checkpoint management), a retrieval layer (FAISS dense index, AST structural index, and fusion), and a deployment layer (FastAPI backend and Streamlit UI). Every layer is unit-tested in isolation, and the full stack is integration-tested end to end via the FastAPI TestClient with fakes standing in for the model and network calls, so the test suite (164 tests as of this report — see Section 5.6) runs in under five seconds with no GPU required.

3.2 Data Pipeline

The data layer downloads and normalizes five sources, each with a dedicated downloader module and a shared caching convention so re-running a notebook does not re-download data already present. CoDocBench supplies paired code-and-documentation triples for the documentation-generation and commit-message-generation tasks. Spider and BIRD supply text-to-SQL question/schema/query triples, each shipped with its own SQLite database archive that the SQL task module introspects at runtime to build schema-aware prompts. CodeParrot supplies the general-purpose retrieval corpus embedded for the RAG pipeline, and a filtered Rust subset of CodeParrot supplies both the fine-tuning corpus and a held-out evaluation set for the language-extension objective. Every dataset passes through a common cleaning and normalization step (whitespace normalization, deduplication, and a length filter) before being split into train/validation/test partitions with a fixed random seed, so results are reproducible across runs.

3.3 Base Model and Task Modules

The model layer wraps Salesforce/codegen-350M-multi behind a single CodeGenModel class exposing two operations: text generation (with a configurable GenerationConfig controlling temperature, top-p, max new tokens, and stop sequences per task) and embedding extraction (used by the retrieval layer, Section 3.5). Five task modules — program synthesis, documentation generation, commit message generation, code translation, and SQL generation — are thin subclasses of a common BaseTask, each responsible only for its task-specific prompt template and output postprocessing, so the generation and evaluation machinery underneath is shared rather than duplicated five times.

3.4 Fine-Tuning Methodology

Two fine-tuning passes extend the base model to Rust, executed in sequence across Checkpoints 1 and 2. The Checkpoint 1 pass trains a LoRA (Low-Rank Adaptation) adapter — a small set of trainable low-rank matrices inserted into the base model's attention layers, leaving the original 350M parameters frozen — on a 500-sample Rust subset, chosen deliberately small to validate the training and checkpointing pipeline quickly before committing to a full run. The Checkpoint 2 pass replaces that subset run with a full-corpus fine-tune over the complete filtered Rust dataset, reusing the identical LoRAFineTuner and CheckpointManager classes so the only variable that changes between the two runs is data volume, not code path. The CheckpointManager additionally implements disk-space-safe pruning — retaining only the best-scoring checkpoints on a Drive volume with finite quota — and a find_resume_point('best') method used by both the RAG pipeline (Section 3.5) and the deployment layer (Section 3.7) to automatically select the strongest available checkpoint for a language rather than requiring a hardcoded path.

Both passes share the same optimizer family (AdamW) and a linear learning-rate schedule with warmup, configured entirely through configs/training.yaml rather than hardcoded in the training script, so a teammate rerunning the notebook can change any hyperparameter without touching code. The LoRA pass targets the attention query and value projection matrices specifically, at a modest rank chosen to keep the adapter's own parameter count a small fraction of the frozen base model — the standard trade-off in

parameter-efficient fine-tuning between adaptation capacity and training/storage cost. Training progress for both passes is logged to TensorBoard (and optionally Weights & Biases) at a fixed step interval, giving a visible loss curve that was used during the project to catch a divergent run early rather than discovering a bad checkpoint only after evaluation.

Key training configuration

- Optimizer: AdamW with linear warmup and decay, learning rate and warmup ratio set in configs/training.yaml (not hardcoded).
- LoRA target modules: attention query and value projections, at a rank sized to keep the adapter small relative to the frozen 350M base parameters.
- Checkpoint cadence: saved at a fixed step interval, pruned automatically to retain only the best-scoring checkpoints given finite Drive quota.
- Resume/selection: CheckpointManager.find_resume_point('best') is the single source of truth used by both offline evaluation and the deployment layer, so training, evaluation, and serving can never silently disagree about which checkpoint is "the" trained model.

3.5 Retrieval-Augmented Generation Pipeline Design

The RAG pipeline embeds a corpus of over 2,000 samples (CodeParrot plus CoDocBench) using the same CodeGenModel.embed() method used elsewhere in the project, builds a FAISS dense vector index over those embeddings, and separately builds an AST structural index over the same corpus by extracting node-type sequences from each code sample's parsed abstract syntax tree. At query time, three retrieval strategies are available: dense-only (semantic similarity via embedding distance), AST-only (structural similarity via node-type sequence overlap), and hybrid (both signals fused via Reciprocal Rank Fusion, which combines two ranked lists by the reciprocal of each item's rank position rather than requiring the two signals' raw scores to be on a comparable scale). A Top-K and strategy sweep (Section 5.2) was run to identify the best-performing configuration before that configuration was used in the four-tier comparison (Section 5.3).

The pipeline is deliberately decoupled from any single model via injected embed_fn and generate_fn callables, documented in the pipeline's own docstring as existing specifically so it can drive different model tiers — small LM, upper-bound LLM without RAG, upper-bound LLM with RAG, and fine-tuned model with RAG — through the identical retrieval and fusion logic. Section 11.1 documents a gap found in how that four-tier design was originally wired at the evaluation layer, and how it was subsequently closed and verified.

3.6 Evaluation Methodology

Every generation task is scored with a combination of exact match, CodeBLEU, and BERTScore F1 (the CodeBERTScore-style metric described in Section 2.7). SQL generation is additionally scored with execution accuracy — does the generated query actually execute against the target database and return the same result set as the gold query, independent of superficial textual differences — rather than text-similarity metrics, since two SQL queries can be functionally identical while being textually very different (differing column order, explicit versus implicit joins, and so on). Program synthesis is additionally scored with pass@k where sampling budget allows. CodeBLEU's structural and dataflow components specifically require a language grammar (via tree-sitter) to compute; where that grammar is unavailable, the metric degrades gracefully to a text-similarity fallback rather than crashing the evaluation run, a design decision

validated by a dedicated regression test (Section 11.1 discusses a related bug where this fallback path was silently active for Python when it should not have been).

Metric definitions used throughout this report

- Exact Match — 1 if the generated output is byte-identical to the reference after whitespace normalization, else 0; averaged over the sample. A strict, low-recall signal reported for completeness rather than as the primary quality measure for open-ended tasks.
- CodeBLEU — a weighted combination of n-gram match, weighted keyword match, AST structural match, and dataflow match (Ren et al., 2020); the structural and dataflow terms require a working tree-sitter grammar for the target language (Section 4.5).
- BERTScore F1 — the harmonic mean of embedding-based precision and recall between generated and reference text, using contextual embeddings rather than surface n-grams; more forgiving of semantically equivalent but textually different output.
- Execution Accuracy (SQL only) — 1 if the generated query executes without error against the target SQLite database and its result set matches the gold query's result set, else 0; averaged over the sample.
- Pass@k (program synthesis, where sampling budget allows) — the probability that at least one of k independently sampled completions passes the task's held-out functional test suite (Chen et al., 2021).

3.7 Deployment Architecture

Deployment exposes the identical task modules and RAG pipeline used in the research notebooks through a FastAPI backend (seven endpoints as of this report — full reference in Appendix A), with a Streamlit UI as a thin HTTP client on top rather than a second implementation of the underlying logic. This split is deliberate: heavy computation (model calls, metric scoring) lives server-side in FastAPI, while Streamlit only renders forms and calls the API client, keeping the UI layer simple and making the API independently testable and independently reusable by any other client. Both services are containerized (separate Dockerfiles for the API and the UI) and orchestrated via docker-compose for local or on-premises deployment, and — as documented in Section 5.6 — an additional, single-container packaging was produced later for durable public hosting on Hugging Face Spaces, combining both services into one container since that platform runs a single container per deployment.

3.8 Delivery Methodology

Beyond the research methodology above, this project also followed a deliberate delivery methodology once the four checkpoints were functionally complete: verify that exploratory sandbox fixes actually reached the team's live Google Drive repository; root-cause and fix any gaps found rather than assume they were cosmetic; grow the automated test suite to lock in every fix; verify each fix live in the actual deployed demo rather than trusting the code alone; publish the verified repository to both a personal and a shared mentor GitHub repository; and consolidate documentation and reporting so the finished work is easy for graders and teammates to review. Sections 4 through 9 report the outcomes of that methodology in detail, including the specific verification evidence for each claim.

4. Outcomes in Stages

The project was executed across four checkpoints, each additive on the last, sharing the same Settings object, model wrapper, and results directory. This section reports what was concretely produced at each stage, followed by the additional post-deployment engineering outcomes delivered afterward.

4.1 Stage 1 (Checkpoint 1) — Foundation, Baseline Evaluation, Rust LoRA

Established the environment bootstrap (Colab and local execution paths, seeded for reproducibility), downloaded CoDocBench and a Rust corpus subset, implemented the five task modules against the untouched base model, and evaluated that base model on four of the five tasks (SQL generation followed in Checkpoint 2 once schema-aware prompting was implemented). This stage also established the project's shared Settings/config pattern (Section 3.1) that every later checkpoint builds on, so no later stage had to re-solve basic questions like where data, checkpoints, and results live on disk.

- Outcome: the project's first fine-tuned checkpoint — a LoRA adapter trained on a 500-sample Rust subset — plus baseline metrics for program synthesis, documentation generation, commit message generation, and code translation.
- Artifacts: configs/*.yaml scaffolding, the five task modules, the first entries in results/comparison_table.csv, and the first saved LoRA checkpoint under checkpoints/.

4.2 Stage 2 (Checkpoint 2) — SQL Generation and Full Fine-Tuning

Downloaded the full Spider and BIRD datasets with their SQLite database archives, implemented schema-aware SQL generation (introspecting each target database's schema and injecting it into the generation prompt), added TensorBoard/Weights & Biases logging for training runs, and replaced the Stage 1 LoRA subset run with a full-corpus Rust fine-tune, including disk-space-safe checkpoint pruning so the growing set of saved checkpoints did not exhaust the available Drive quota.

- Outcome: a full-corpus Rust fine-tuned checkpoint and real execution-accuracy scores for text-to-SQL generation on both Spider and BIRD (Section 5.1).
- Artifacts: the SQL task module's schema-introspection logic, the full-corpus fine-tuning script, TensorBoard/W&B run logs, and the pruned, best-checkpoint-retaining checkpoints/ directory used by every later stage.

4.3 Stage 3 (Checkpoint 3) — Retrieval-Augmented Generation

Embedded a corpus of over 2,000 samples, built a FAISS dense index and a separate AST structural index, and swept retrieval strategy (dense, AST, hybrid) and Top-K (1, 3, 5, 10, and a dynamic setting) to find the best-performing configuration, then ran the four-tier comparison (small LM baseline, upper-bound LLM without RAG, upper-bound LLM with RAG, and fine-tuned model with RAG) using that configuration.

- Outcome: the best retrieval configuration identified from the sweep (AST strategy, Top-K = 1, static rather than dynamic retrieval count — full results in Section 5.2), and the project's first real, non-placeholder four-tier comparison table (Section 5.3).
- Artifacts: faiss_index/ (the persisted dense vector index), the AST structural index, results/topk_sweep.csv, and the first version of the four-tier comparison logic in src/codegen_rag/rag/topk_experiment.py — later revisited and fixed in Section 4.5 / 11.1.

4.4 Stage 4 (Checkpoint 4) — Deployment

Exposed every task module and the RAG pipeline through a FastAPI backend and a demo Streamlit UI, containerized and orchestrated via docker-compose, with API documentation auto-generated at /docs (Swagger UI) and /redoc, and a Docker healthcheck on both services.

- Outcome: a live, containerized, tested deployment — the same demo used for the final presentation and, as documented in Section 5.5, subsequently exposed publicly and demonstrated live with real user queries.
- Artifacts: src/codegen_rag/api/ (FastAPI app and routes), src/codegen_rag/app/ (Streamlit UI and API client), docker/Dockerfile.api and docker/Dockerfile.streamlit, and the docker-compose.yml orchestrating both.

4.5 Post-Deployment Engineering Outcomes

Two further engineering gaps were identified and closed during deployment hardening, beyond the original four-checkpoint scope. First, checkpoint-aware model routing: every deployment endpoint had been serving the base model regardless of whether a trained Rust checkpoint existed, because the checkpoint-resolution logic checked for a checkpoint's marker files one directory level too shallow (the checkpoint manager nests every save under a checkpoint-NNNNNN subdirectory). This was corrected with an explicit resolution layer (_best_checkpoint_dir, _resolve_run_checkpoint, _is_valid_checkpoint in src/codegen_rag/api/dependencies.py) that locates the best available checkpoint per language, covered by eight dedicated unit tests. Second, genuine AST/dataflow-aware CodeBLEU: without a compatible tree-sitter-python grammar installed, CodeBLEU's structural components were silently falling back to a plain text-similarity approximation for every Python evaluation in the project — a regression that would not raise an error, only quietly report a less meaningful number. Adding tree-sitter-python at a version verified compatible with the pinned tree-sitter and codebleu package versions restored real structural scoring, confirmed by a dedicated regression test checking that a syntactically restructured-but-equivalent snippet scores differently from a naive text-similarity baseline would predict.

- Outcome: deployment automatically serves the best available fine-tuned Rust checkpoint (confirmed live via the Streamlit demo), and CodeBLEU scores reflect genuine structural similarity rather than a silent text-similarity fallback. Test coverage grew from 108 to 147 passing tests to lock in both fixes.

5. Final Outcomes

5.1 Baseline Comparison Table

The table below reports the small-LM baseline tier's results from the project's own comparison_table.csv, generated by the Checkpoint 3 evaluation run — the first real, non-placeholder numbers produced once the notebooks were run end to end in Colab with a GPU. All metrics are shown as percentages, matching the presentation used in the live Streamlit demo.

Task	Model Tier	N	Exact Match	CodeBLEU	BERTScore F1	Exec. Accuracy
sql_generation_spid	small_lm_baseline	200	—	—	—	7.00%

Task	Model Tier	N	Exact Match	CodeBLEU	BERTScore F1	Exec. Accuracy
er						
sql_generation_bird bench	small_lm_baseline	200	—	—	—	0.00%
program_synthesis	small_lm_baseline	100	0.00%	10.46%	87.62%	—
commit_message_ge neration	small_lm_baseline	100	0.00%	0.02%	81.67%	—
code_translation	small_lm_baseline	29	0.00%	4.24%	81.94%	—
documentation_gen eration	small_lm_baseline	100	0.00%	2.17%	—	—

5.2 Retrieval Strategy Sweep Results

Before the four-tier comparison in Section 5.3 could be run, a sweep across retrieval strategy (dense, AST, hybrid) and Top-K (1, 3, 5, 10, and a dynamic retrieval count) was run to identify the best-performing retrieval configuration, evaluated by CodeBLEU on a fixed sample of 30 queries per configuration.

Strategy	Top-K	Dynamic	N	CodeBLEU
dense	1	False	30	8.11%
dense	3	False	30	7.64%
dense	5	False	30	7.89%
dense	10	False	30	9.31%
dense	dynamic	True	30	7.86%
ast	1	False	30	9.89%
ast	3	False	30	8.26%
ast	5	False	30	9.57%
ast	10	False	30	7.97%
ast	dynamic	True	30	8.03%
hybrid	1	False	30	7.38%
hybrid	3	False	30	8.04%
hybrid	5	False	30	6.21%
hybrid	10	False	30	8.24%

Strategy	Top-K	Dynamic	N	CodeBLEU
hybrid	dynamic	True	30	8.00%

The best configuration found was AST-strategy retrieval with Top-K = 1 (CodeBLEU 9.89%), narrowly ahead of AST at Top-K = 5 (9.57%) and dense at Top-K = 10 (9.31%). The general pattern — AST-only retrieval outperforming both dense-only and hybrid fusion at low Top-K — suggests that for this corpus and query style, a single highly structurally relevant example is often more useful to the generator than several semantically related-but-structurally-mismatched examples, though hybrid fusion narrows that gap at Top-K = 3 (8.04% vs. AST's 8.26%). This configuration (AST, Top-K = 1) was carried forward into the four-tier comparison below.

5.3 Four-Tier Comparison Results

The table below reports the four-tier comparison — small LM baseline, upper-bound LLM without RAG, upper-bound LLM with RAG, and the fine-tuned Rust checkpoint with RAG — each on the same fixed sample of 15 queries, scored by CodeBLEU.

Model Tier	N	CodeBLEU
small_lm_baseline	15	7.26%
llm_no_rag	15	24.95%
llm_rag	15	17.55%
fine_tuned_rag	15	7.52%

The upper-bound LLM without RAG scores highest (24.95%), roughly 3.4x the small baseline. Adding RAG to the upper-bound LLM actually reduces its score on this sample (17.55%) rather than improving it — a result worth stating plainly rather than hiding, and discussed further in Section 5.4. The fine-tuned-plus-RAG tier (7.52%) scores close to, but distinguishably above, the small baseline (7.26%), which is itself an important verification result: this number being distinct from (not identical to) the baseline is the concrete evidence that the fine-tuned model is genuinely being used in this tier rather than silently falling back to the base model, closing the gap described in Section 11.1.

5.4 Discussion

The near-zero exact-match scores across every task in Section 5.1 are expected for open-ended code and text generation, where many valid solutions exist for a single prompt and an exact string match is a poor proxy for correctness. Program synthesis's CodeBLEU (10.46%) and BERTScore F1 (87.62%) are the small baseline's strongest showing among the text-similarity metrics. Commit message generation's very low CodeBLEU (0.02%) alongside a comparatively high BERTScore F1 (81.67%) is a concrete illustration of why both metrics are reported together rather than either alone: commit messages are short, free-form natural language with no consistent structural pattern for CodeBLEU's AST-oriented components to match against, so a semantically reasonable message can still score near zero on CodeBLEU while scoring respectably on a semantic-similarity metric like BERTScore. Both SQL benchmarks in Section 5.1 show low execution accuracy at the small-LM baseline tier (7.00% Spider, 0.00% BIRD), matching the literature's

expectation (Section 2.5) that BIRD's real-world database complexity is substantially harder than Spider's cleaner cross-domain schemas.

The RAG result in Section 5.3 — retrieval reducing rather than improving the upper-bound LLM's score — deserves explicit interpretation rather than being reported without comment. A capable model with strong prior code knowledge can be pulled off-course by retrieved context that is topically related but not precisely on point for the specific query, especially on a small 15-query sample where a small number of unhelpful retrievals can measurably move the average. This is a genuine, informative negative result rather than a bug: it demonstrates that retrieval augmentation is not universally beneficial and that its value is conditional on retrieval quality and on how much the base generator already knows, which is exactly the kind of finding a controlled four-tier comparison is designed to surface. A larger evaluation sample and a qualitative review of individual retrieved-versus-not-retrieved generations would be the natural next step to characterize this effect more precisely (see Section 12).

5.5 Deployment Verification

The final deployed system was verified live, not merely assumed correct from passing tests. The FastAPI backend's health endpoint and the Streamlit demo UI both confirm the Rust-language endpoint is served by the fine-tuned checkpoint rather than the base model. The demo was additionally exposed publicly via an ngrok tunnel and exercised with real, ad hoc user queries across every tab — program synthesis, translation, SQL generation, and RAG — with live screenshots captured showing correctly formatted percentage metrics and a working live per-query scoring panel (Section 11.3) returning real CodeBLEU, BERTScore, and exact-match numbers for a user-typed query against a user-typed reference, not a precomputed dataset average. The full 164-test pytest suite passes in under five seconds with no GPU, network access, or downloaded model weights required.

5.6 Publishing Outcomes

The verified, 164-test-passing repository was published to two GitHub locations: a personal repository (github.com/mahin-aeroai/CodeGen-) and the mentor-shared collaboration repository (github.com/nayanjha16/CodeGen-Implementations-May_26) on its Group-39 branch, with the full project documentation consolidated into a fully rewritten README.md reflecting the project's true, current state — a live demo link, the closed gaps, and the live-scoring feature — rather than the stale pre-delivery-phase description that previously existed there. Publishing to the mentor repository specifically required diagnosing and resolving a 403 permission-denied push failure, root-caused to an insufficiently scoped Personal Access Token rather than a missing collaborator grant, documented fully in Section 7.5.

Separately, because the Colab-plus-ngrok demo setup only stays reachable while a Colab runtime remains actively connected — Google Colab enforces session limits on the order of hours, not the days or weeks a grader might want to revisit a live link — a durable, always-on deployment path was identified and packaged for this project: a combined single-container Docker build (docker/Dockerfile.spaces and docker/start.sh) suitable for Hugging Face Spaces, which runs continuously without requiring an open notebook, alongside step-by-step deployment instructions.

6. Testing and Quality Assurance

Because the development sandbox used to write this project's delivery-phase fixes had no GPU and no live API access (Section 7.6), correctness had to be established through disciplined automated testing rather

than through manually re-running the full pipeline after every change. This section documents that testing approach as a first-class part of the project's methodology, not an afterthought.

6.1 Test Suite Architecture

The 164-test pytest suite is organized to mirror the six-layer system architecture described in Section 3.1: dedicated test modules for the data layer (downloaders, preprocessing, splitting), the model layer (generation and embedding, with a fake model standing in for the real 350M-parameter network), the five task modules, the training layer (LoRA and full fine-tune configuration, checkpoint save/prune/resume logic), the RAG layer (index building, each retrieval strategy, fusion, and the RAGAugmentedTask adapter), the evaluation layer (every metric, including the CodeBLEU fallback-versus-real-structural-scoring distinction from Section 4.5), and the deployment layer (every FastAPI endpoint via TestClient, and the checkpoint-resolution helpers from Section 7.3). No test in the suite requires a GPU, network access, or a downloaded model checkpoint — every external dependency is replaced with a fake or fixture, which is what keeps the full suite runnable in under five seconds in any environment, including the constrained sandbox this project's delivery-phase work was actually done in.

6.2 Test Categories and Coverage

- Unit tests — the majority of the suite; one behavior per test, one assertion focus per test, covering both the happy path and documented edge cases (empty input, missing checkpoint, unsupported language, malformed schema).
- Integration tests — exercise multiple layers together, most notably the full FastAPI request/response cycle for every endpoint in Appendix A via TestClient, and the RAG pipeline's retrieval-then-generation-then-scoring flow end to end with fakes.
- Regression tests — added specifically alongside each bug fix in this report (checkpoint routing, CodeBLEU fallback, Checkpoint 4 RAG wiring) so the exact failure mode that was found and fixed cannot silently reappear in a future change without a test failing first.
- Contract tests — verify that the Streamlit UI's API client sends and parses requests/responses in the shape the FastAPI backend actually expects, catching the class of bug where a UI change and a backend change silently drift apart.

6.3 Continuous Verification Discipline

Every patch delivered during the delivery phase followed the same discipline: write or extend the relevant test(s) first, confirm the test fails against the un-patched code (so it is known to actually exercise the bug), apply the fix, confirm the full suite passes, and only then treat the fix as complete. This discipline is what makes the specific claims in Sections 4.5, 11.1, and 11.3 of this report — that a routing bug was fixed, that a scoring fallback was closed, that a RAG-integration gap was closed — checkable rather than asserted: each is backed by a named test file and a passing-count delta in the table below, not merely a description of intended behavior.

6.4 Test Suite Growth Timeline

The table below traces the suite's growth across the delivery phase, tying each increment to the specific fix or feature described elsewhere in this report.

Milestone	Passing Tests	Delta	Reason
End of Checkpoint 4 (pre-delivery)	108	—	Baseline research-phase suite covering data, model, tasks, training, RAG, and API layers.
Repository sync patch	147	+39	Combined MD5-verified sync patch reconciling a divergent sandbox copy with the team's live Drive repository across 10 files.
Checkpoint-routing + CodeBLEU fixes	153	+6	Eight new tests for checkpoint-aware model resolution plus a regression test for genuine AST-aware CodeBLEU scoring, net of minor consolidation.
Checkpoint 4 RAG gap closure	159	+6	Six regression tests covering the new fine_tuned_generate_fn parameter and the RAGAugmentedTask adapter.
Live scoring feature	164	+5	New tests for the /score and /score_sql endpoints, client methods, and percentage-formatting UI logic (net of shared fixtures).

7. Challenges

Deploying and iterating on this system live in Google Colab surfaced several infrastructure and process issues beyond ordinary application bugs, each worth documenting because the symptoms were, in every case, initially indistinguishable from a code-level regression.

7.1 Google Drive FUSE Write-Sync Race

A file write to a Drive-mounted path can succeed and self-verify within the same Colab session without having actually flushed to persistent storage, and a Runtime restart shortly afterward can silently revert it — a fix would appear to work, then reappear broken after any restart, with no error message at any point. This was root-caused through repeated MD5 re-verification across restarts, and fixed by rewriting every deployment patch cell to retry its write with a short delay and a full read-back-and-hash comparison rather than treating a single write as final. This retry-with-verification pattern was then reused across every subsequent patch delivered in this project's delivery phase, so it never had to be re-diagnosed.

7.2 Stale Compiled Bytecode on a Drive-Mounted Package

Google Drive's FUSE mount can report file modification times unreliably, causing Python to keep using an old compiled .pyc file even after the corresponding source file was confirmed correct on disk — surfacing as an ImportError for a function demonstrably present in the source, which is a particularly confusing failure mode since the obvious first debugging step (re-reading the source file) shows nothing wrong.

Fixed by recursively deleting every __pycache__ directory before the next Runtime restart, forcing Python to recompile from the actual current source.

7.3 Checkpoint Directory Nesting

The checkpoint-routing feature initially checked for a checkpoint's marker files one directory level too shallow, because the checkpoint manager nests every save under a checkpoint-NNNNNN subdirectory rather than saving directly into the named checkpoint folder. This meant every deployment endpoint silently served the base model even when a valid trained checkpoint existed on disk, with no error or warning — arguably the most consequential bug found in this project, since it would have quietly invalidated the entire premise of the fine-tuning objective (Section 1.2) at the deployment layer specifically, while leaving the training and evaluation notebooks themselves unaffected. Fixed with an explicit checkpoint-resolution helper and eight dedicated unit tests (Section 4.5).

7.4 Repository Path and Access Issues

A repo-path mismatch was discovered during the delivery-phase repository audit: the project's own settings.root_dir resolved to a different Drive folder (CodeGen_Capstone) than the actual git-tracked source repository (codegen-rag-capstone), so trained checkpoints, the FAISS index, processed data, and evaluation results all lived in a location the git-based publishing workflow was not looking at. This was discovered by explicitly loading the project's Settings object and printing settings.root_dir rather than assuming the two paths matched, and every subsequent artifact-handling script (including the deployment packaging in Section 5.6) was written to pull code and trained artifacts from their two separate confirmed-correct locations rather than assuming a single shared folder.

7.5 GitHub Token Scope and Push Permissions

A push to the mentor-shared repository initially failed with a 403 error — "Permission denied" — which was, at first, reasonably suspected to be a missing collaborator grant on the target account, since the same error persisted across three structurally different push methods (a git-init-based approach, a fresh-clone-based approach, and an interactive getpass-token approach identical to a previously working method). What ultimately resolved it was a detail easy to miss when generating a classic GitHub Personal Access Token: the token had been created without its top-level repo scope checkbox checked, so it authenticated correctly as the right account but was not authorized to push to any repository regardless of that account's actual collaborator status. Regenerating the token with the repo scope explicitly checked resolved the push immediately. This is documented here specifically because the failure mode (identical 403 message, regardless of whether the real cause is account permissions or token scope) makes the two causes easy to conflate, and ruling out the wrong one first can waste significant debugging time.

7.6 Scope and Resource Constraints

The development environment used to write and test the code had no GPU, no live API access, and no practical way to download the multi-gigabyte datasets — every module was instead unit- and integration-tested in isolation with fakes standing in for real model and network calls, with the first true end-to-end numbers produced only once the notebooks were run in Colab with a GPU and valid API keys. This constraint shaped the project's engineering discipline directly: because no local run could ever produce a real number, correctness had to be argued through test coverage and code review rather than "it worked when I ran it," which in turn is why the test suite's growth (108 to 164 passing tests over the delivery phase) is reported as a first-class outcome throughout this report rather than a footnote.

7.7 Inherent Small-Model Quality Limits

Separate from any bug, the small base model's raw generation quality on open-ended prompts is genuinely limited — observed directly in the live demo, where a plain-language prompt for a simple, well-known algorithm could produce output that drifted into unrelated boilerplate rather than a correct implementation. This is not a defect to fix; it is the expected, and in fact the entire point of, this project's four-tier comparison design (Section 1.2): the small baseline tier exists specifically to be outperformed by the fine-tuned, upper-bound-LLM, and RAG-augmented tiers, and the live per-query scoring feature added in the delivery phase (Section 11.3) makes this gap directly visible and quantifiable for any query a user chooses to type, rather than only for the fixed evaluation sample in comparison_table.csv.

8. Applicability in the Real World

The techniques demonstrated in this project map directly onto problems organizations face today when adopting AI-assisted software development, beyond their role as an academic exercise.

8.1 Cost-Efficient Developer Tooling

Small, self-hosted code models are dramatically cheaper to run at scale than proprietary frontier LLMs — no per-token API cost, no data leaving the organization's own infrastructure, and predictable compute cost rather than usage-based billing. This project's four-tier comparison methodology (small baseline vs. fine-tuned vs. upper-bound LLM vs. upper-bound LLM with RAG) is directly reusable by any organization deciding whether a smaller, self-hosted model is good enough for a specific internal use case, and by how much a targeted fine-tune or a retrieval layer could close any remaining gap before committing to the ongoing cost of a proprietary API.

Concretely, a platform team evaluating this trade-off could reuse this project's evaluation harness (Section 3.6) directly: point it at their own internal tasks, run the same four tiers, and get a defensible, quantified answer to "how much would we lose by self-hosting" rather than a qualitative guess — which is exactly the gap this project was designed to fill in Section 1.1's framing of the underlying business question.

8.2 Extending Coverage to Underrepresented Languages

The Rust fine-tuning workflow generalizes directly to other underrepresented or organization-specific languages: legacy enterprise languages (COBOL, Fortran) still running critical production systems, internal domain-specific languages with no public training data at all, or newer languages not yet well represented in public pretraining corpora. The same LoRA-then-full-fine-tune progression, checkpoint management, and automatic best-checkpoint routing used here for Rust apply unchanged to any of those targets.

This matters most for organizations sitting on large legacy codebases in a language current-generation code models were never trained on in depth — the LoRA-first, full-fine-tune-second progression demonstrated in Section 3.4 gives such an organization a low-cost way to validate the approach on a small subset before committing GPU budget to a full-corpus run.

8.3 Natural-Language Database Access

The text-to-SQL capability directly supports letting non-technical stakeholders query databases in plain English, evaluated the way that matters for safety in a production setting: execution accuracy — does the generated query actually run and return the correct result — rather than superficial query-text similarity,

which is exactly the distinction that would matter if this were deployed as a self-service analytics tool inside an organization rather than left purely as a benchmark score.

8.4 Retrieval-Augmented Generation Grounded in Proprietary Code

The hybrid dense-plus-AST retrieval pipeline is directly transferable to grounding a code assistant in an organization's own private codebase, reducing hallucinated APIs and outdated patterns without retraining the underlying model — an organization would simply re-embed its own repository in place of the CodeParrot/CoDocBench corpus used here, with no change to the retrieval or fusion logic itself. Section 5.4's negative RAG result is directly relevant here too: it is a concrete demonstration that retrieval augmentation needs to be evaluated on the target domain rather than assumed beneficial by default.

8.5 Developer Productivity Automation

Documentation-generation and commit-message-generation directly automate two of the most commonly deprioritized developer chores, both evaluated here against a realistic dataset (CoDocBench) built from real commit history rather than synthetic examples, giving a more honest picture of how such a tool would perform in practice.

8.6 A Reusable Deployment Blueprint

The FastAPI-plus-Streamlit-plus-Docker deployment pattern, with checkpoint-aware model routing and a documented API surface (Appendix A), is a directly reusable blueprint for standing up an internal developer-tooling service — and, as demonstrated in Section 5.6, the same codebase can additionally be repackaged into a single-container build suitable for a managed, always-on hosting platform, without changing any application logic.

9. Risk Register and Project Management

This section records the project's risk management approach as an explicit artifact rather than leaving it implicit in the narrative elsewhere in this report — every risk below either closes with a specific, checkable fix referenced by section number, or is carried forward transparently as an open item.

9.1 Risk Register

Risk	Likelihood	Impact	Mitigation	Status
Sandbox fixes never reach the live Drive repository	High	High	Explicit repository audit before every delivery-phase claim of completion; MD5-verified patch cells (Section 7.1).	Closed
Trained checkpoint silently unused at deployment	Medium	High	Checkpoint-aware routing with 8 dedicated unit tests (Section 4.5, 7.3).	Closed
CodeBLEU silently degrades to a text-	Medium	Medium	Pinned compatible tree-sitter-python; regression test asserting	Closed

Risk	Likelihood	Impact	Mitigation	Status
similarity proxy			structural sensitivity.	
Checkpoint 4 RAG-integration requirement not actually met	High	High	Code audit against the official brief; explicit fine_tuned_generate_fn parameter and RAGAugmentedTask adapter; verified live (Section 11.1).	Closed
GitHub push blocked by permissions	Medium	Medium	Root-caused to PAT scope rather than account access; documented decision process (Section 7.5) for future recurrence.	Closed
Public demo link goes offline (Colab session limits)	High	Medium	Durable single-container Hugging Face Spaces packaging prepared as an always-on alternative (Section 5.6).	Mitigated, not yet confirmed live
Evaluation sample sizes too small to generalize (e.g., 15-query four-tier comparison)	Medium	Low	Explicitly flagged in Section 5.4 discussion and Section 12 future work rather than over-claiming from a small sample.	Open, documented
Reference-list citation errors undermine grader confidence	Low	Low	Two citation-year errors and one unverified citation identified and corrected/flagged during report authoring (Section 2).	Closed / one item open

9.2 Project Timeline and Milestones

Checkpoint	Focus	Key Artifact Delivered
Checkpoint 1	Foundation, baseline evaluation, Rust LoRA	Five task modules, baseline metrics for four tasks, first LoRA checkpoint
Checkpoint 2	SQL generation, full fine-tuning	Schema-aware SQL generation, full-corpus Rust checkpoint, Spider/BIRD execution accuracy
Checkpoint 3	Retrieval-augmented generation	FAISS + AST retrieval, strategy/Top-K sweep, first real four-tier comparison table
Checkpoint 4	Deployment	FastAPI + Streamlit, Dockerized, tested

Checkpoint	Focus	Key Artifact Delivered
		end to end
Delivery phase (post-Checkpoint 4)	Verification, gap closure, publishing	164-test suite, Checkpoint 4 gap closed and verified live, live scoring feature, two GitHub repositories published, durable-hosting package

9.3 Tools and Technology Stack

- Modeling and training: PyTorch, Hugging Face Transformers, PEFT (LoRA), Salesforce/codegen-350M-multi.
- Retrieval: FAISS (dense vector search), tree-sitter (AST parsing), a custom reciprocal-rank-fusion implementation.
- Evaluation: CodeBLEU, CodeBERTScore-style BERTScore, custom SQL execution-accuracy harness against SQLite.
- Deployment: FastAPI, Streamlit, Docker / docker-compose, Hugging Face Spaces (Docker SDK) for durable hosting.
- Testing and QA: pytest, FastAPI TestClient, fixture-based fakes for model and network calls.
- Delivery infrastructure: Google Colab, Google Drive (FUSE-mounted), ngrok, GitHub (two repositories), TensorBoard / Weights & Biases.

10. Team Contributions

This section records what the team delivered, both at the checkpoint level across the four original research milestones and, in detail, during the delivery and publishing phase that followed.

10.1 Team Contributions (Checkpoints 1–4)

The four-checkpoint research pipeline — environment bootstrap, dataset integration, task modules, model fine-tuning, the retrieval-augmented generation pipeline, and the FastAPI/Streamlit deployment — was built collaboratively by the full team (Mahin Nandipa, Abhinaya Thavishi, and Ashu Bagul) across the project's four checkpoint milestones, under the guidance of mentors Pawan Baswani and Nayan Anand and supervisor Prof. Anil Neelkanti. Commit history on the shared mentor repository's Group-39 branch reflects ongoing contributions from multiple team members. Per the project proposal, checkpoint ownership across research, implementation, evaluation, deployment, documentation, and testing is collective rather than assigned per person.

Coordination across the four checkpoints followed the milestone cadence set out in Section 9.2: each checkpoint's scope was agreed with mentors ahead of the milestone, implemented and reviewed as a team, and evaluated against the project's shared results directory before moving to the next stage, so that Checkpoint 2's SQL work built on Checkpoint 1's task-module conventions rather than reinventing them, Checkpoint 3's retrieval layer built on Checkpoint 2's fine-tuned checkpoint rather than a fresh model, and Checkpoint 4's deployment exposed the exact task modules and RAG pipeline validated in Checkpoint 3 rather than a reimplementation.

10.2 Delivery-Phase Contributions

Beyond the original four checkpoints, the team carried out a further delivery and publishing phase, working from the existing four-checkpoint codebase to verify, repair, test, deploy, and publish it, and to prepare the final academic deliverables. The table below summarizes that work.

Area	Contribution
Repository audit	Identified that exploratory sandbox fixes had not reached the team's live Google Drive repository, and that the repository's settings.root_dir pointed to a different Drive folder than its actual git-tracked source, requiring every subsequent fix and artifact-handling step to target the confirmed-correct paths (Section 7.4).
Test suite growth	Diagnosed a 22-test gap by diffing pytest collection output against the team's own verbose run, then delivered a combined, MD5-verified sync patch for 10 files, growing the suite from 108 to 147 passing tests; subsequent gap-closure and feature work grew it further to 164.
Checkpoint-aware model routing	Root-caused why deployment endpoints were silently serving the base model instead of the trained Rust checkpoint (a directory-nesting bug, Section 7.3), and implemented the fix plus its eight dedicated unit tests.
Genuine AST-aware CodeBLEU	Identified that CodeBLEU's structural scoring was silently falling back to a plain text-similarity proxy, and resolved it by pinning a compatible tree-sitter-python version, with a regression test to prevent recurrence.
Infrastructure diagnostics	Root-caused two further non-obvious Colab/Drive issues — a file write-sync race and stale compiled bytecode (Sections 7.1–7.2) — and established the retry-with-verification pattern used across every subsequent patch cell delivered this project.
Missing .gitignore and LICENSE	Discovered that the repository's .gitignore was completely absent, added a replacement and cleaned __pycache__ contamination out of git history, and added an MIT LICENSE file.
Checkpoint 4 requirement gap — root-caused and closed	Audited the project against the official brief and found that the "put the fine-tuned model into the RAG system and evaluate" requirement had no working path in the codebase; implemented the fix (explicit fine_tuned_generate_fn parameter and a new RAGAugmentedTask adapter) plus six regression tests, then verified live that the fine_tuned_rag tier's real number is distinct from the baseline (Section 5.3, Section 11.1).
Live per-query scoring feature	Designed and implemented two new API endpoints (/score, /score_sql), client methods, and Streamlit UI changes so any query typed into the demo can be scored instantly against a user-supplied reference, with percentage formatting applied throughout the UI; covered by 11 new tests and verified live in the deployed demo.
GitHub publishing	Diagnosed and resolved a 403 push failure to the personal repository

Area	Contribution
	(Section 7.5) and published the verified repository there.
Mentor repository publishing	Diagnosed the same class of 403 failure recurring against the mentor repository, isolated it specifically to GitHub token scope rather than account permissions after three different push methods failed identically, and published the verified repository to the mentor-shared collaboration repository's Group-39 branch.
README consolidation and currency	Rewrote the project README to reflect the system's true, current state — a live demo link, the closed Checkpoint 4 gap, and the live-scoring feature — replacing a stale description that predated the delivery phase, on both the personal and mentor repositories.
Durable deployment path	Identified that the Colab-plus-ngrok demo link only stays reachable while a Colab runtime remains connected, and packaged a combined single-container Docker build for Hugging Face Spaces (always-on hosting) as a durable alternative, with full deployment instructions.
Final deliverable authoring	Authored and formatted this report, the accompanying Presentation Slides deck, and the Research Paper Summaries document, restructuring all three to match the official capstone submission rubric and correcting two citation errors found in the process (Section 2).

11. Limitations and Gap-Closure Note

This repository was built to be fully correct and runnable, but portions of it were not executed against a GPU, the live Claude Sonnet 4 / GPT-5 APIs, or the multi-gigabyte datasets inside the development sandbox used to write and test the code. Every component was instead unit- or integration-tested in isolation with fakes standing in for real model and network calls (164 of 164 tests passing as of this report, up from 108 at the start of the delivery phase). The results in Sections 5.1–5.3 come from real, non-placeholder end-to-end Colab runs with a GPU and, where applicable, live API keys.

11.1 Checkpoint 4 Gap — Closed and Verified

An audit of this project against the official brief found that Checkpoint 4's explicit requirement to "put their model into the RAG system and evaluate" had no working path in the codebase: run_four_tier_comparison() in src/codegen_rag/rag/topk_experiment.py reused the base model's generate function inside its fine-tuned-model branch, so a "fine-tuned + RAG" tier would have silently just re-run the base model; and the only RAG comparison path scored CodeBLEU alone into a table separate from the project's actual results/comparison_table.csv, so even a correct run would not have reached the deliverable. Both issues were fixed: run_four_tier_comparison() now takes an explicit fine_tuned_generate_fn parameter (with a logged warning if omitted, so the old failure mode can never happen silently again), and a new RAGAugmentedTask adapter (src/codegen_rag/rag/rag_task_adapter.py) lets any task module be retrieval-augmented and scored through the same Evaluator.evaluate_task() path as every other tier, landing full

exact_match/CodeBLEU/BERTScore metrics in comparison_table.csv. Six new regression tests cover both fixes.

Unlike at the time of an earlier draft of this report, this fix is no longer merely designed but has now been run end to end and verified: the fine_tuned_rag row in the live four-tier comparison table (Section 5.3) shows 7.52% CodeBLEU, a real, distinct number from the small baseline's 7.26% — concrete evidence the fine-tuned checkpoint is genuinely being exercised inside the RAG comparison, not silently substituted with the base model as it previously was.

11.2 Spider vs. Spider 2.0 — Resolved

See Section 2.9: the brief's reference link points to Spider 2.0, but the project's actual code (verified directly in src/codegen_rag/data/downloaders.py) downloads and evaluates against the classic 2018 Spider benchmark, matching the project's own README citation. This is documented as a near-certain reference-list error in the brief rather than an implementation gap, with a one-line mentor confirmation recommended rather than a rebuild.

11.3 Live Per-Query Scoring — Delivered

A live per-query scoring feature was added during the delivery phase: any query typed into the Streamlit demo, on any task tab, can be scored instantly against a user-supplied reference via two new endpoints (POST /score for text-similarity tasks, POST /score_sql for SQL execution-match), rather than only ever showing the fixed dataset-level averages in comparison_table.csv. This was verified live — a screenshot captured during this delivery phase shows a real fibonacci-series query scored at 0.00% exact match, 12.64% CodeBLEU, and 85.00% BERTScore F1 against a user-typed reference solution, confirming the feature computes real numbers for an arbitrary query rather than a precomputed placeholder.

11.4 Public Demo Availability — Delivered, With a Durability Caveat

The deployed demo was made publicly reachable via an ngrok tunnel during the delivery phase and demonstrated live. The specific limitation worth stating plainly: a free-tier ngrok tunnel, and the Colab runtime hosting it, do not stay available indefinitely — Colab enforces session time limits regardless of activity, so this specific link will eventually go offline until the notebook is re-run. A durable alternative (a single-container Docker build for Hugging Face Spaces, Section 5.6) was packaged specifically to remove this limitation, but as of this report has not yet been confirmed deployed and running — it is delivered as a ready-to-run set of files and instructions rather than a live, already-verified second URL.

11.5 Remaining Open Item

The "AST retrieval (Jiang et al., 2023)" citation in the project README remains unverified against a specific paper and should be confirmed against the original project proposal before external submission.

12. Conclusion and Future Work

This project delivers a complete, tested, end-to-end pipeline for evaluating and enhancing a small open-source code language model, and — beyond the original research scope — a fully synchronized, tested, verified-in-production, and published repository ready for mentor and grader review. The delivery phase closed the Checkpoint 4 RAG-integration requirement gap and verified that closure with a real, distinct result in the live comparison table; corrected two functional bugs (checkpoint-aware model serving and

genuine AST/dataflow-aware CodeBLEU scoring); root-caused three non-obvious Colab/Google-Drive infrastructure issues and a GitHub token-scope permissions issue; added a new live per-query scoring feature to the deployed demo; grew the test suite from 108 to 164 passing tests; and produced a complete publishing trail across two GitHub repositories plus a packaged path to durable, always-on public hosting.

Natural next steps include confirming the Hugging Face Spaces deployment is actually running (Section 11.4) rather than only packaged; completing an end-to-end run large enough to populate the upper-bound-LLM and LLM+RAG rows of comparison_table.csv at full evaluation scale rather than the 15-query sample used in Section 5.3; qualitatively reviewing individual retrieved-versus-not-retrieved generations to characterize the negative RAG result in Section 5.4 more precisely; reproducing and resolving the PEFT "missing adapter keys" warning observed when loading the Rust LoRA checkpoint; confirming the Spider-vs-Spider-2.0 scope with mentors (Section 11.2); resolving the unverified AST-retrieval citation (Section 11.5); and, longer term, extending the fine-tuning approach demonstrated here for Rust to additional underrepresented languages, and piloting the retrieval-augmented and deployment blueprint described in Section 8 against a real internal codebase rather than a general-purpose public corpus.

Acknowledgments

This project would not have reached a deployed, tested, and published state without the guidance of mentors Pawan Baswani and Nayan Anand, whose feedback across all four checkpoints shaped both the technical scope and the standard of rigor applied to it, and supervisor Prof. Anil Neelkanti, whose oversight anchored the project to the capstone program's requirements throughout. Thanks are also due to the broader Group 39 cohort for the collaborative environment in which this work was built, and to the maintainers of the open datasets and open-source tools this project depends on directly — CodeParrot, CoDocBench, Spider, BIRD, Hugging Face Transformers and PEFT, FAISS, tree-sitter, FastAPI, and Streamlit — without which a project of this scope would not have been feasible within a single academic term.

13. References

Nijkamp, E., et al. (2022). CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis.

Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. (Introduces both the Codex model and the pass@k metric.)

Feng, Z., et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages.

Pai, K. S., et al. (2024). CoDocBench: A Dataset for Code-Documentation Alignment in Software Maintenance. (Corrected from "Pal et al." in the project README.)

Yu, T., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.

Li, J., et al. (2023). Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD). NeurIPS 2023. (Corrected from "2024" in the project README.)

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

Lu, S., et al. (2021). ReACC: A Retrieval-Augmented Code Completion Framework.

Jiang, et al. (2023). AST-based code retrieval. Citation unverified against a specific publication — see Section 11.5.

Ren, S., et al. (2020). CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.

Zhou, S., et al. (2023). CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code.

(pass@k is introduced within Chen et al., 2021, above, rather than a separate publication.)

Appendix A: API Endpoint Reference

Full interactive documentation is auto-generated at /docs (Swagger UI) and /redoc for the running FastAPI service. The table below summarizes every endpoint referenced elsewhere in this report.

Endpoint	Method	Purpose
/generate	POST	Program synthesis from a natural-language problem description, served by the checkpoint-aware routed model.
/document	POST	Documentation generation for a given code snippet.
/translate	POST	PL-to-PL code translation between a source and target language.
/sql	POST	Natural-language-to-SQL generation with schema-aware prompting for a named database.
/rag	POST	Retrieval-augmented generation with a selectable strategy (dense / ast / hybrid), Top-K, and an optional upper-bound LLM toggle.
/score	POST	Live per-query scoring (exact match, CodeBLEU, BERTScore F1) of a prediction against a user-supplied reference. Added in the delivery phase (Section 11.3).
/score_sql	POST	Live execution-match scoring of a predicted SQL query against an optional gold query. Added in the delivery phase (Section 11.3).
/health	GET	Liveness/readiness check used by the Docker healthcheck and the Streamlit demo's status panel.

Appendix B: Repository Structure

Top-level layout of the codegen-rag-capstone repository, as published to both GitHub locations described in Section 5.6:

- configs/ — all tunables (project, data, model, training) as YAML, no magic numbers in code.

- src/codegen_rag/ — config loader, utils, data downloaders/preprocessors, model wrapper, task modules, sql schema handling, rag pipeline (including the RAGAugmentedTask adapter), evaluation metrics/Evaluator, training (LoRA/full fine-tune + checkpoint management), api (FastAPI app), and app (Streamlit UI + API client).
- notebooks/ — 01_checkpoint1_foundation_and_docgen.ipynb through 04_checkpoint4_deployment.ipynb, each additive on the last.
- scripts/ — CLI equivalents for running evaluation and serving the API outside Colab.
- tests/ — the 164-test pytest suite, requiring no GPU, network access, or downloaded model weights.
- docker/ — Dockerfile.api, Dockerfile.streamlit (docker-compose two-service deployment), plus Dockerfile.spaces and start.sh added in the delivery phase for single-container hosting (Section 5.6).
- docs/architecture.md — Mermaid system and sequence diagrams.
- README.md — consolidated project documentation, rewritten during the delivery phase to reflect the system's current, verified state.

Appendix C: Sample Queries and Model Outputs

The examples below are drawn from live exercising of the deployed demo during the delivery phase (Sections 5.5, 11.3), not from the fixed evaluation datasets, and illustrate concretely what the metrics in Section 5 represent in practice.

C.1 Program Synthesis — Small Baseline vs. Live Scoring Panel

Prompt: "Write a Python function that returns the first n numbers of the Fibonacci series."

Small-LM baseline behavior: on this style of open-ended prompt, the untuned 350M base model frequently drifts away from a correct implementation partway through generation, illustrating the small-model quality limit discussed in Section 7.7 rather than a defect in the surrounding system.

Live per-query scoring result (Section 11.3, verified live): scoring a generated candidate against a user-supplied correct reference implementation through the new /score endpoint returned Exact Match 0.00%, CodeBLEU 12.64%, BERTScore F1 85.00% — concretely showing how a generation can be semantically in the right neighborhood (a respectable BERTScore) while being structurally and token-wise quite different from the reference (low CodeBLEU, zero exact match).

C.2 Text-to-SQL Generation

Natural-language question: "List the names of all students enrolled in more than three courses."

The SQL task module introspects the target database's schema at request time and injects it into the prompt (Section 3.6), then the generated query is scored by execution accuracy — run against the real SQLite database and compared by result set — rather than by text similarity to a gold query, since two queries with different column ordering or explicit-vs-implicit joins can be equally correct.

C.3 Retrieval-Augmented Generation

Query: a request for a utility function in the style of the embedded CodeParrot/CoDocBench corpus. Under the AST, Top-K = 1 configuration identified as best in Section 5.2, the pipeline retrieves the single

structurally closest code sample from the corpus and includes it as in-context grounding before generation, rather than retrieving several topically related but structurally mismatched examples — the sweep result in Section 5.2 is the empirical basis for that specific configuration choice.

C.4 Documentation Generation

Input: an undocumented Python function from the CoDocBench evaluation split. Output is scored against the real, human-written documentation paired with that function in CoDocBench's commit-history-mined dataset (Section 2.4), rather than against a synthetic or templated reference, which is why documentation-generation scores in Section 5.1 are a meaningfully harder, more realistic test than a templated benchmark would provide.

Appendix D: Environment Setup and Reproducibility Guide

D.1 Prerequisites

- Python 3.10+, pip, and (for real end-to-end runs) a CUDA-capable GPU — Google Colab's free or Pro GPU runtime is sufficient for every experiment reported in this document.
- API keys for the upper-bound LLM comparison tiers (Claude Sonnet 4, with GPT-5 configured as an automatic fallback), stored as environment variables or Colab secrets — never committed to the repository.
- A GitHub Personal Access Token with the top-level repo scope checked, only required for publishing changes back to a remote (Section 7.5).

D.2 Running the Research Notebooks

- Clone the repository and open notebooks/01_checkpoint1_foundation_and_docgen.ipynb through 04_checkpoint4_deployment.ipynb in order — each is additive on the state produced by the previous one, sharing the same Settings object and results directory (Section 3.1).
- Install dependencies from the repository's requirements file inside the notebook's first cell before running anything else.
- Run pytest from the repository root before trusting any notebook output — the 164-test suite (Section 6) requires no GPU or network access and should complete in under five seconds.

D.3 Running the Deployment Locally

- docker-compose up from the docker/ directory starts both the FastAPI backend and the Streamlit UI as two containers, wired together via the compose network.
- Visit /docs on the API container for interactive Swagger documentation of every endpoint in Appendix A, and the Streamlit UI's own URL for the demo interface.
- For durable public hosting instead of a local or Colab-tunneled deployment, use docker/Dockerfile.spaces and docker/start.sh with Hugging Face Spaces (Docker SDK), which combine both services into a single container listening on port 7860 (Section 5.6).

D.4 Reproducibility Notes

- Every data split uses a fixed random seed (Section 3.2), so re-running the data pipeline produces identical train/validation/test partitions.
- Checkpoint selection is always resolved through CheckpointManager.find_resume_point('best') (Section 3.4) rather than a hardcoded path, so evaluation and deployment can never silently diverge on which trained checkpoint is being used.
- All tunables live in configs/*.yaml rather than in code, so a reviewer can see every configuration decision made for a given run without reading the implementation.